

ANÁLISIS Y DISEÑO DE APLICACIONES II

UNIDAD TEMÁTICA 5 – ESTILOS DE ARQUITECTURA

TRABAJO FINAL DE UNIDAD

Luca Bessonart, Santiago Trinidad, Santiago Acosta, Santiago Severo

PARTE 1

En esta entrega, “evolucionaremos” nuestra aplicación de pedidos desarrollada en los trabajos de fin de unidad anteriores, desde un monolito modular hacia una arquitectura de microservicios y una operación migrada de REST a SOAP.

1. Identificación y justificación del estilo arquitectónico

Estilo de la entrega anterior:

El Software que hemos desarrollado en las TFU anteriores, sigue un estilo monolítico. Toda la funcionalidad de nuestro sistema, como lo puede ser el catálogo, clientes, pedidos, descuentos, etc., vive en un único proyecto, aunque dentro del mismo, los componentes están separados en interfaces.

A la vez, en el despliegue, tenemos dos contenedores de la API (api 1, api2) dentro de Nginx, y se utiliza una sola Base de Datos (PostgreSQL) compartida. Este aspecto, alguien externo a nuestro equipo de trabajo, podría mencionar y confundirse de que es una arquitectura distribuida, pero no lo es, ya que las dos réplicas de las apis, son copias idénticas del mismo sistema modular (monolito). Cada réplica sabe hacer todo, pero esta técnica de replicación sirve para distribuir la carga, pero no cambia el estilo.

Limitaciones de la entrega anterior:

Por otro lado, durante los tests realizados a la aplicación de la TFU4, analizamos y encontramos ciertos problemas como por ejemplo, qué sucede si nuestra BD se cae. Si PostgreSQL se cae, se derrumba toda la aplicación, ya que no se puede consultar ni el catálogo, ni los clientes, ni los pedidos, por ejemplo.

Estilo Seleccionado:

Adicionalmente, teniendo en cuenta estas limitaciones, para esta TFU, hemos decidido migrar nuestra solución a una arquitectura de microservicios. En este estilo, la aplicación se dividirá en servicios de tamaño menor, cada uno engloba una capacidad de negocio distinta. Cada servicio se despliega de forma independiente.

Razones que fundamentan el cambio:

Desde la TFU3, nuestra aplicación se encuentra organizada en componentes con interfaces definidas, donde intentamos marcar y diferenciar con ellas contextos de negocios claros.

A la vez, queremos que una falla no tire abajo todo el sistema, sino que buscamos mejorar el atributo de calidad de Disponibilidad. Con servicios y bases separadas, si el catálogo se cae, el resto de las funciones, pueden seguir disponibles.

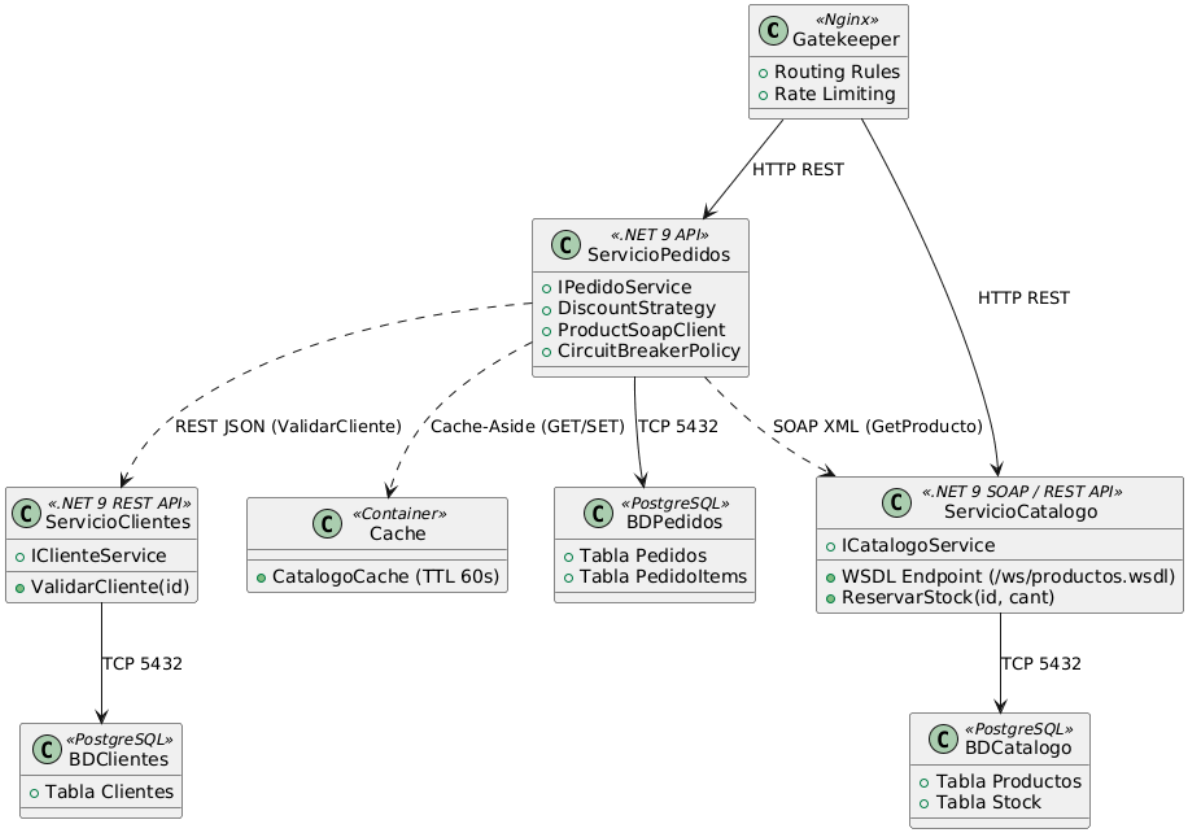
Estilos que descartamos inicialmente:

En primer lugar, descartamos la arquitectura basada en eventos, ya que inicialmente nos obliga a trabajar con consistencia eventual y creemos que no permitirá al cliente saber si su pedido queda confirmado o no, y también requiere agregar un broker como intermediario.

Por otro lado, descartamos también la arquitectura orientada a servicios (SOA), ya que teniendo en cuenta lo visto en clase y la información recabada en la web, SOA está pensada para organizaciones de gran tamaño, con grandes aplicaciones. Para nuestro caso, nuestra aplicación es muy pequeña, y creemos que la solución en si, seria muy rebuscada y creemos que nos provocaría ciertos fallos.

2. Modelado de la nueva arquitectura

a. Diagrama de Componentes



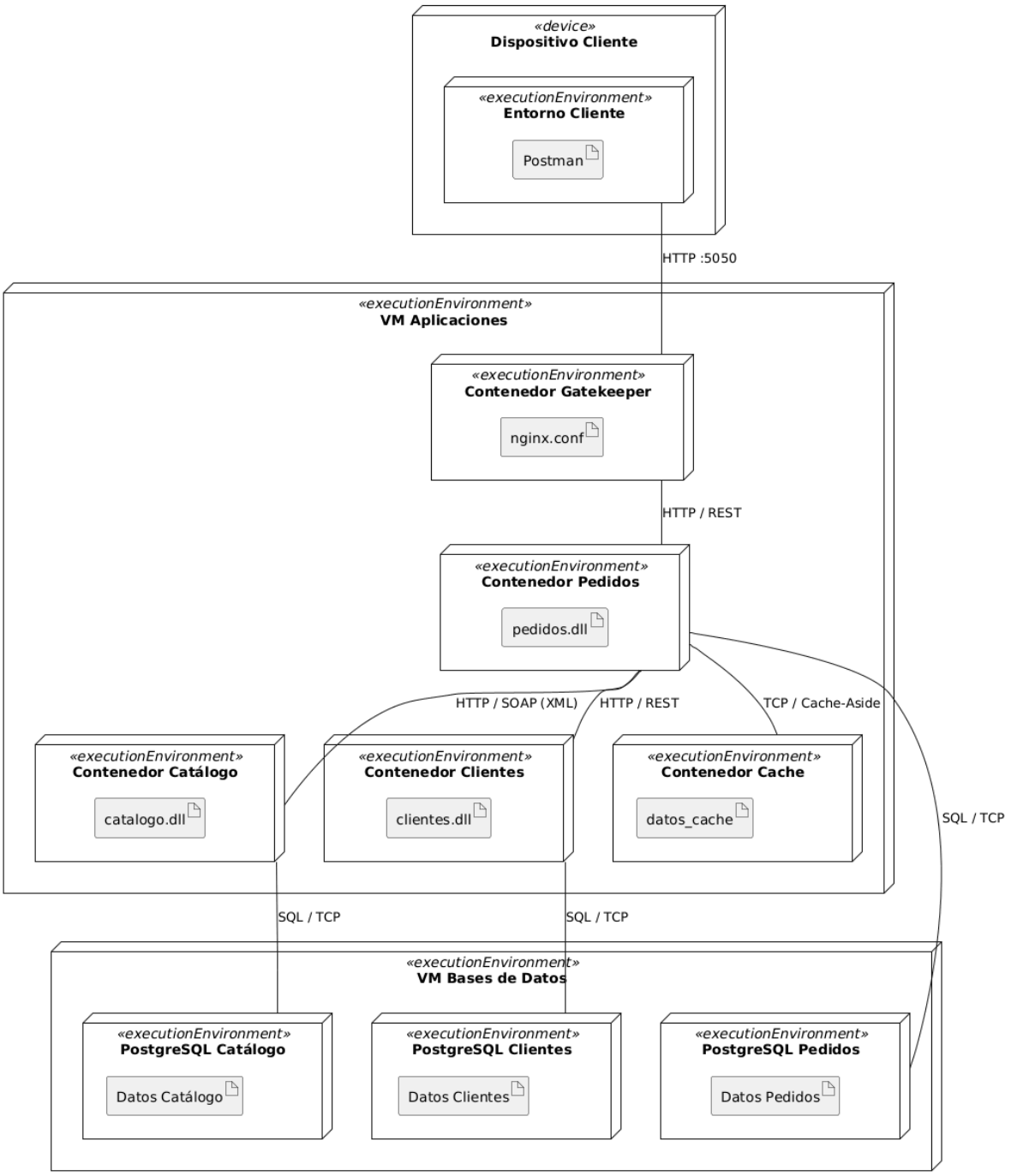
Este diagrama modela la estructura lógica del sistema bajo una arquitectura de microservicios, donde cada componente engloba un contexto de negocio independiente con interfaces claramente delimitadas:

- Gatekeeper (Nginx): Va a actuar como proxy inverso y será el punto de entrada unificado para el sistema. Expone reglas de ruteo con Routing Rules y de control de frecuencia de solicitudes con Rate Limiting, garantizando que la red interna de microservicios permanezca aislada y protegida de tráfico malicioso o desmedido.
- Servicio de Pedidos (ServicioPedidos): Encapsula la lógica central de creación y gestión de órdenes de compra (IPedidoService) e integra las estrategias de descuento intercambiables (DiscountStrategy). Para interactuar de forma segura con el catálogo, incorpora un cliente SOAP

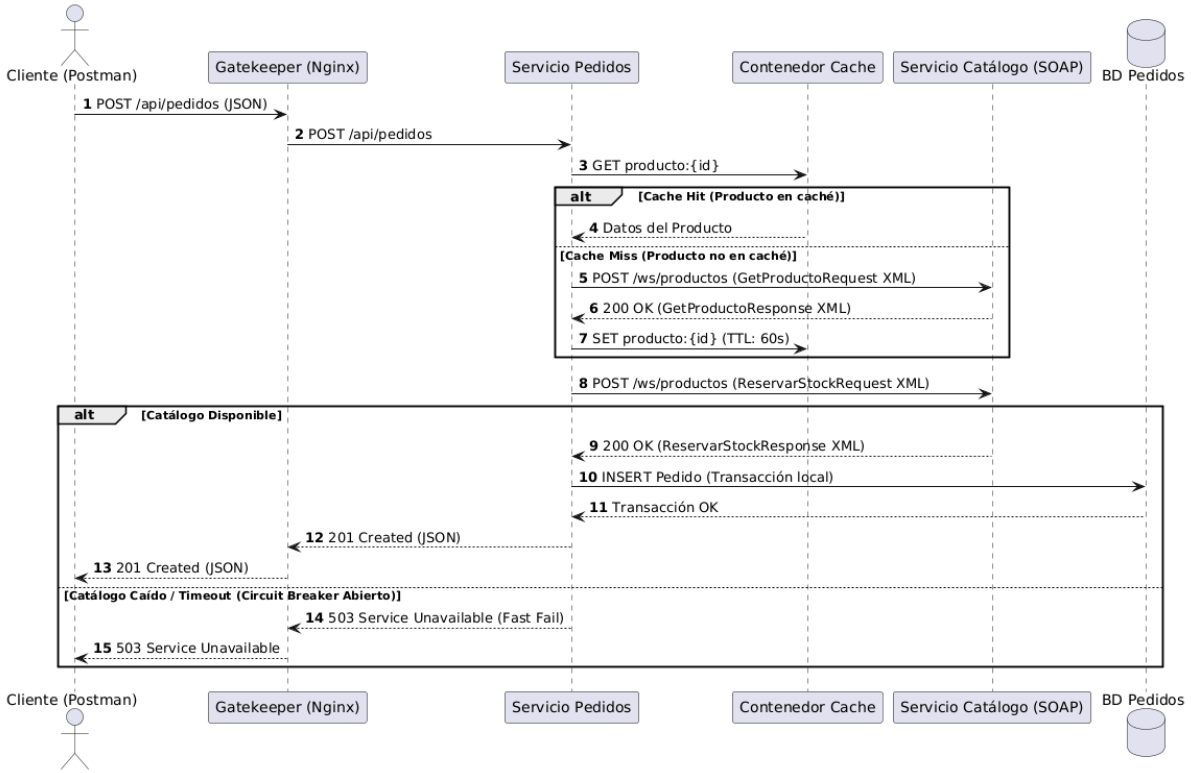
(ProductSoapClient) respaldado por un patrón de tolerancia a fallos (CircuitBreakerPolicy), previniendo fallas en cascada si la red o el servicio externo fallan.

- Servicio de Catálogo (ServicioCatalogo): Ofrece la gestión de productos e inventario. Publica un contrato WSDL explícito en el endpoint " /ws/productos.wsdl para habilitar operaciones vía mensajes SOAP XML (como la consulta de detalles con GetProducto), además de gestionar la actualización y reserva de inventarios mediante la operación ReservarStock (id, cant).
- Servicio de Clientes (ServicioClientes): Microservicio autónomo dedicado a la administración y verificación de clientes (IClienteService), exponiendo la interfaz REST con datos en formato JSON a través del método ValidarCliente(id).
- Contenedor de Caché (CacheContainer): Aplica el patrón Cache-Aside manteniendo en memoria temporal (CatalogoCache) los datos de productos leídos con mayor frecuencia, configurado con un tiempo de vida (TTL) de 60 segundos para reducir la sobrecarga computacional y la latencia entre servicios.

b. <u>Diagrama de Despliegue</u>

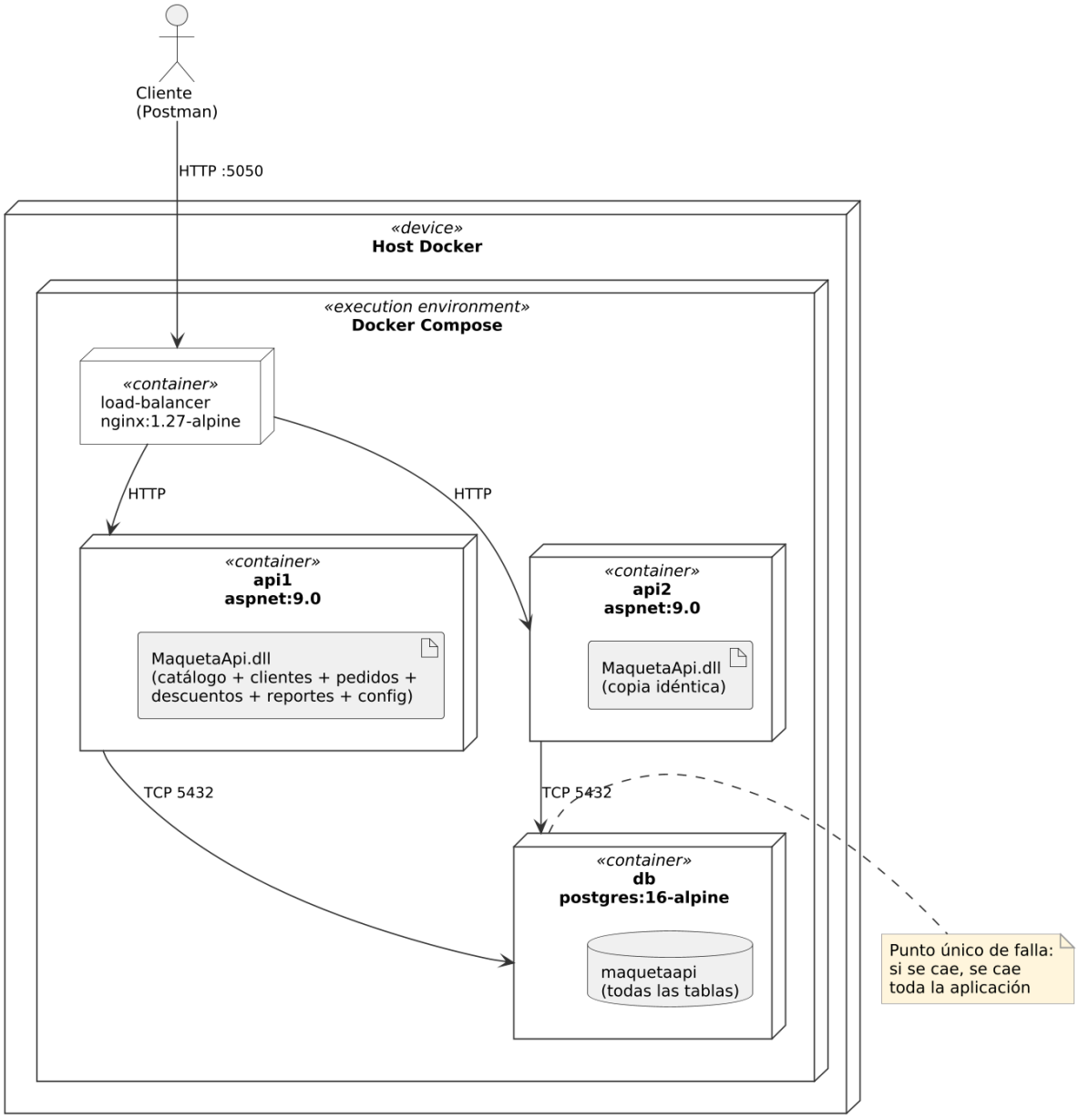


c. Diagrama de Secuencia: Crear un Pedido


Cliente (Postman)
Gatekeeper (Nginx)
Servicio Pedidos
Contenedor Cache
Servicio Catálogo (SOAP)
BD Pedidos
1 POST /api/pedidos (JSON)
2 POST /api/pedidos
3 GET producto:{id}
alt
[Cache Hit (Producto en caché)]
4 Datos del Producto
[Cache Miss (Producto no en caché)]
5 POST /ws/productos (GetProductoRequest XML)
6 200 OK (GetProductoResponse XML)
7 SET producto:{id} (TTL: 60s)
8 POST /ws/productos (ReservarStockRequest XML)
alt
[Catálogo Disponible]
9 200 OK (ReservarStockResponse XML)
10 INSERT Pedido (Transacción local)
11 Transacción OK
12 201 Created (JSON)
13 201 Created (JSON)
[Catálogo Caído / Timeout (Circuit Breaker Abierto)]
14 503 Service Unavailable (Fast Fail)
15 503 Service Unavailable
Cliente (Postman)
Gatekeeper (Nginx)
Servicio Pedidos
Contenedor Cache
Servicio Catálogo (SOAP)
BD Pedidos

3. Comparación entre estilos arquitectónicos

El cambio a microservicios mejora el aspecto mencionado anteriormente, fallas, la escalabilidad y el despliegue independiente de los distintos contenedores.



Nota: Figura generada con IA para contemplar el sistema monolito desplegado en la TFU4

Diferencias Principales entre arquitecturas

	UT4: Monolito	UT5: Microservicios
Responsabilidades	Un solo proceso hace todo, los componentes están separados únicamente por interfaces a nivel de código.	Cada servicio tiene una responsabilidad distinta a nivel de negocio (catalogo, clientes o pedidos).
BD	Una Base de Datos compartida, donde cualquier componente puede leer o escribir cualquier tabla.	Una Base de Datos por servicio, donde cada uno solamente accede a sus propios datos, y los demás datos que se necesiten, los solicita a traves de la API.
Comunicación	Llamadas a métodos	REST con JSON entre la mayoria de los servicios y SOAP con XML, para el servicio de ObtenerProducto.
Consistencia al Crear un Pedido	Transacción ACID, o se hace todo, o no se hace nada	Transacciones y consistencia eventual en el procesamiento de pedidos.
Despliegue	Nginx, dos replicas identicas y	RELLENAR

	una BD.	
Escalabilidad	Se puede replicar el monolito completo.	Se replica solo el servicio que se necesita, ejemplo catálogo que es lo más leido.
Disponibilidad	La BD es un punto de falla, si se cae, se cae el sistema completo.	Una falla queda contenida solamente en el servicio que dejó de funcionar. Las funcionalidades que no dependen de él, siguen funcionando.
Rendimiento	Muy buena	Quizas peor, crear un pedido implica varias llamadas y una de ella con XML (SOAP).
Modificabilidad y Despliegue	Cualquier cambio, obliga a desplegar todo.	Un cambio en un servicio se despliega sin tocar a los demás, si y solo si, no cambian los contratos de negocio.

Pros y Contras de cada alternativa

Monolito (TFU4)

A favor:

- Es más sencillo de desarrollar, desplegar y luego depurar.
- Tiene consistencia fuerte gracias a las transacciones vistas en clase ACID.
- En cuanto al atributo de calidad Rendimiento, tiene mejor latencia por la comunicación entre componentes.

En contra:

- La BD compartida podría ser un único punto de falla.
- No permite escalar cada parte por separado
- Cualquier cambio obliga a redesplegar todo.

Microservicios (TFU5):

A favor:

- Aisla las fallas por servicio.
- Permite escalar y desplegar cada servicio por separado.
- Al tener cada uno su base, se elimina el acoplamiento por datos.

En contra:

- Perdemos la única transacción.
- Existe latencia entre servicios
- Existen más “piezas” para desplegar, monitorear y testear.
- Existe la posibilidad de que algún dato quede repetido.

Compromisos asociados a la elección

- Al elegir microservicios, garantizamos que una falla no tire abajo la disponibilidad del servicio, sino que se busca consistencia inmediata.

- Al tener que migrar el metodo obtenerProducto de REST a SOAP, esperamos que al viajar tambien el XML, la creacion del pedido sea más lento que la TFU4, lo vamos a medir.

-

4. **Falacias de la computación distribuida y patrones arquitectónicos**

1. **La red es confiable**

Para crear un pedido, el servicio de Pedidos depende de llamar por red al Servicio de Clientes, a la operación SOAP ObtenerProducto de catálogo y a la reserva de stock de Catálogo.

Cualquiera de estas llamadas puede fallar. El contenedor de catálogo puede estar caido, o la respuesta puede no llegar nunca o puede tener retardo.

¿Qué problemas podría ocacionar?

- Si el catálogo se cuelga, cada solicitud de Pedidos quedaría esperando la respuesta. Las solicitudes se acumulan, Pedidos se queda sin recursos y termina dejando de responder también, y se generaría una falla en cascada.

- Si la reserva de stock se realiza pero la respuesta se pierde en el camino, el módulo de Pedidos podría creer que falló, e intentará realizarla nuevamente. Si simplemente reintentara, el stock se descontará dos veces, lo cual genera inconsistencias en el stock.

Decisiones de diseño y patrones que aplicamos:

1. Timeout en todas las llamadas entre servicios.

2. Circuit Breaker en las llamadas de Pedido hacia Catálogo.

3. Retry

4. Operaciones idempotentes. Como la reserva y la expedición del pedido llevan el pedidoID, se pueden repetir sin efectos duplicados. Esto es lo que hace que hace seguro y consistente a el Retry.

Elegimos estos patrones porque solamente con Retry no alcanza. Si el catálogo está caído durante un minuto, retry no lo arregla. El Circuit Breaker resuelve este problema, ya que corta las llamadas mientras que el servicio esté caído.

En la TFU 4 habíamos descartado CB ya que no había red entre componentes, pero ahora sí podría ser coherente.

Falacia 2: “La Latencia es cero”

En la TFU 4, consultar el catálogo era una llamada dentro del mismo proceso. Ahora, crear un pedido implica una llamada a Clientes, una llamada SOAP por cada ítem del pedido, una llamada para reservar stock y la escritura en la base de pedidos. Cada llamada por la red suma milisegundos, y la llamada SOAP además tiene que armar y leer un mensaje XML, lo cual genera unos ms de latencia.

¿Qué problemas podría ocasionar?

- Si una llamada se pone lenta, toda la solicitud del cliente se vuelve lenta.
- Las demoras en las llamadas síncronas SOAP hacia Catálogo bloquean los hilos de ejecución en el Servicio de Pedidos, pudiendo dejar al servicio sin recursos para responder otras solicitudes.
- En el caso de un pedido incluya varios artículos y se realice una llamada por red individual para cada uno, la latencia total del proceso se multiplica exponencialmente

Decisiones de Diseño y patrones que aplicamos

1. Caché - Aside en el servicio de Pedidos para obtenerProducto. Antes de llamar a Catálogo, Pedidos buscará el producto en su caché. Si lo encuentra, no hace la llamada SOAP. Si no, llama a Catálogo y guarda la respuesta. El stock nunca se almacena en caché porque lo confirma Catálogo al reservar.

2. Timeouts, para que una llamada lenta no haga esperar a todo el sistema indefinidamente.

¿Por qué elegimos estos patrones?

Cache-Aside ya lo usamos en la UT4, así que conocemos su comportamiento. Además, el catálogo es un caso ideal, se lee mucho y cambia poco.

Por otro lado, si un precio cambia en Catálogo, Pedidos puede seguir usando el precio anterior durante hasta 60 segundos. Para nuestro dominio lo consideramos aceptable, y por eso el tiempo de vida es corto. También hay que tener en cuenta que, si la caché responde, no se llama a SOAP.

5. **Atributo de Calidad : Disponibilidad**

El atributo que esperamos mejorar es la disponibilidad, evitando posibles fallas. Es decir, queremos evitar que la caída de un servicio no deje sin funcionar al resto del sistema, y que el sistema se recupere solo cuando el servicio vuelve.

Elegimos este atributo en especial porque es el beneficio principal que los microservicios ofrecen frente al monolito, según lo visto hasta ahora en el curso.

¿Cómo contribuyen las decisiones de arquitectura en nuestra solución?

Timeouts en la llamadas entre servicios.

Táctica: Detectar fallas

Esto permite que Pedidos se de cuenta de que Catálogo no responde por 5 segundos, en vez de esperar indefinidamente.

Circuit Breaker